RAPORT POCZĄTKOWY

System generowania grafów infrastruktury energetycznej z wykorzystaniem modeli LLM

Stan przygotowań i koncepcja prototypu

Temat pracy

Implementacja modeli LLM wspierających infrastrukturę energetyczną z użyciem baz grafowych

Materiał na spotkanie z promotorem
21 października 2026

Cel dokumentu

Dokument podsumowuje decyzje podjęte na początku pracy inżynierskiej oraz przedstawia proponowany kierunek implementacji. Głównym zadaniem systemu będzie przekształcanie tekstowych opisów infrastruktury elektroenergetycznej w ustrukturyzowane grafy przechowywane w Neo4j. Pierwsza wersja prototypu wykorzysta sieć pandapower case14 jako źródło danych referencyjnych.

Najważniejszym kolejnym krokiem jest zatwierdzenie modelu grafowego i minimalnego zakresu danych. Po tej decyzji można zaimplementować deterministyczny konwerter pandapower do Neo4j, a następnie uruchomić niezależną ścieżkę ekstrakcji informacji z tekstu przez LLM.

Zakres pracy

- Ocena zdolności modeli językowych do rozpoznawania elementów infrastruktury, ich właściwości i wzajemnych połączeń.
- Opracowanie procesu konwersji wyekstrahowanych informacji do grafu właściwościowego.
- Implementacja prototypu łączącego Python, modele LLM, walidację danych i Neo4j.
- Wizualizacja grafu oraz podstawowa analiza topologii i połączeń.
- Porównanie grafów wygenerowanych przez LLM z grafem referencyjnym utworzonym bez udziału modelu.

Stan przygotowań

Obszar	Aktualne ustalenie	Status
Dane startowe	pandapower.networks.case14()	ustalone
Baza grafowa	Neo4j i model property graph	ustalone
Backend	Python	ustalone
Integracja LLM	Wymienny adapter dla API oraz modeli lokalnych przez Ollama	koncepcja
Format pośredni	JSON zgodny ze schematem i walidowany przez Pydantic	koncepcja
Ewaluacja	Porównanie z grafem referencyjnym oraz analiza błędów	koncepcja
Implementacja	Kod prototypu i eksperymenty	do wykonania

Dane wejściowe pandapower case14

Sieć case14 stanowi niewielki, kontrolowany przykład, który pozwala sprawdzić pełny proces bez wprowadzania na początku dużego zbioru danych. Dane są przechowywane w tabelach pandas DataFrame. Każdy rekord może zostać odwzorowany na węzeł grafu, a kolumny wskazujące szyny umożliwiają utworzenie relacji.

Tabela	Liczba	Węzeł	Główne odwołania
net.bus	14	Bus	indeks 0-13, name 1-14, geo
net.line	15	Line	from_bus, to_bus
net.trafo	5	Transformer	hv_bus, lv_bus
net.gen	4	Generator	bus
net.load	11	Load	bus
net.ext_grid	do potwierdzenia	ExternalGrid	bus
net.shunt	do potwierdzenia	Shunt	bus

Indeks rekordu i pole name mają inne znaczenie. Dla szyn indeks DataFrame przyjmuje wartości od 0 do 13, natomiast name zawiera oznaczenia od 1 do 14. Relacje trzeba tworzyć na podstawie indeksów używanych w kolumnach from_bus, to_bus, hv_bus, lv_bus i bus. Nazwa powinna pozostać właściwością opisową.

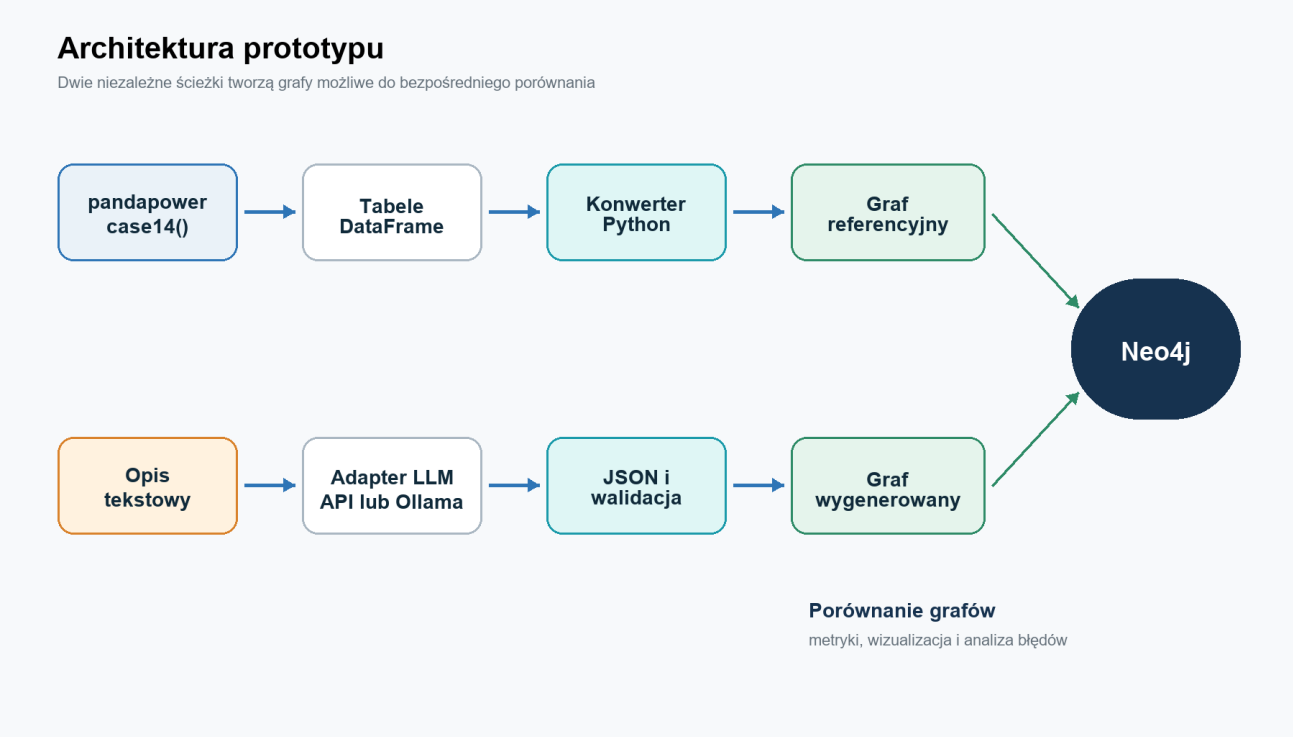
Przed implementacją należy przypiąć wersję pandapower w pliku zależności. Pozwoli to zachować powtarzalność wyników i uniknąć różnic w strukturze danych między wersjami biblioteki.

Minimalny zakres pierwszej wersji

- Wczytanie tabel bus, line, trafo, gen i load oraz sprawdzenie ext_grid i shunt.
- Zachowanie oryginalnego indeksu pandapower w polu pp_index.
- Utworzenie stabilnego identyfikatora source_id zawierającego nazwę sieci, typ elementu i indeks.
- Przeniesienie parametrów wymaganych do identyfikacji, wizualizacji i pierwszych eksperymentów.
- Oddzielenie danych wejściowych od wyników obliczeń rozpływu mocy, takich jak res_bus i res_line.

Architektura systemu

Prototyp będzie zawierał dwie ścieżki generowania grafu. Pierwsza przekształci dane pandapower bezpośrednio do Neo4j i utworzy wynik referencyjny. Druga przetworzy opis tekstowy przez model LLM, zapisze wynik w JSON, zweryfikuje go i dopiero wtedy utworzy graf. Obie ścieżki powinny korzystać z tego samego modelu domenowego.



Rysunek 1 Proponowana architektura prototypu

Odpowiedzialność komponentów

Komponent	Odpowiedzialność
Moduł pandapower	Wczytuje sieć referencyjną i udostępnia tabele DataFrame.
Konwerter referencyjny	Mapuje rekordy tabel na węzły, właściwości i relacje Neo4j.
Adapter LLM	Zapewnia wspólny interfejs dla zewnętrznego API i lokalnej Ollamy.
Warstwa promptów	Przekazuje reguły ekstrakcji, schemat odpowiedzi i przykłady few shot.
Walidator	Sprawdza strukturę JSON, typy, jednostki, identyfikatory i relacje.
Repozytorium Neo4j	Tworzy graf wyłącznie na podstawie zweryfikowanych obiektów.
Moduł ewaluacji	Porównuje graf wygenerowany z referencyjnym i zapisuje błędy.

Proces tworzenia grafu referencyjnego

Graf referencyjny powstanie klasycznym skryptem Pythona. LLM nie uczestniczy w tym procesie, dlatego wynik może służyć jako punkt odniesienia podczas ewaluacji.

- Wczytanie net = pandapower.networks.case14().
- Odczyt wybranych tabel i kolumn oraz normalizacja brakujących wartości.
- Utworzenie węzłów Bus przed elementami wskazującymi szyny.
- Utworzenie węzłów Line, Transformer, Generator, Load i pozostałych urządzeń.
- Utworzenie relacji na podstawie indeksów pandapower.
- Sprawdzenie liczby węzłów i relacji oraz zapis manifestu importu.

Zasada identyfikacji

Każdy element powinien mieć stabilny identyfikator niezależny od pola name. Proponowany format source_id zapobiega kolizjom między typami elementów i między różnymi sieciami testowymi.

```
case14:bus:0
case14:line:0
case14:trafo:0
case14:load:0
```

Kontrole jakości importu

Kontrola	Oczekiwany wynik
Liczba szyn	14 węzłów Bus
Liczba linii	15 węzłów Line
Liczba transformatorów	5 węzłów Transformer
Liczba generatorów	4 węzły Generator
Liczba odbiorów	11 węzłów Load
Integralność relacji	Każda relacja wskazuje istniejący węzeł Bus
Unikalność	Brak powtórzonych source_id w obrębie etykiety

Proces generowania grafu z opisu

Model językowy nie powinien bezpośrednio wykonywać zapytań zapisujących dane. Jego zadaniem będzie przygotowanie propozycji struktury w kontrolowanym formacie. Backend pozostanie odpowiedzialny za reguły biznesowe, walidację i transakcję Neo4j.

Etap	Działanie	Możliwy błąd
1	Użytkownik wprowadza opis infrastruktury	Brak danych koniecznych do odtworzenia grafu
2	Prompt przekazuje schemat i reguły ekstrakcji	Niejednoznaczna instrukcja lub za mało przykładów
3	LLM identyfikuje encje, właściwości i relacje	Pominięcie, duplikat lub halucynacja
4	Model zwraca dane zgodne z JSON Schema	Błąd składni albo niedozwolony typ
5	Pydantic i reguły domenowe sprawdzają wynik	Nieistniejący identyfikator lub zła jednostka
6	Warstwa repozytorium zapisuje graf	Naruszenie constraint lub błąd transakcji

Wstępny kontrakt JSON

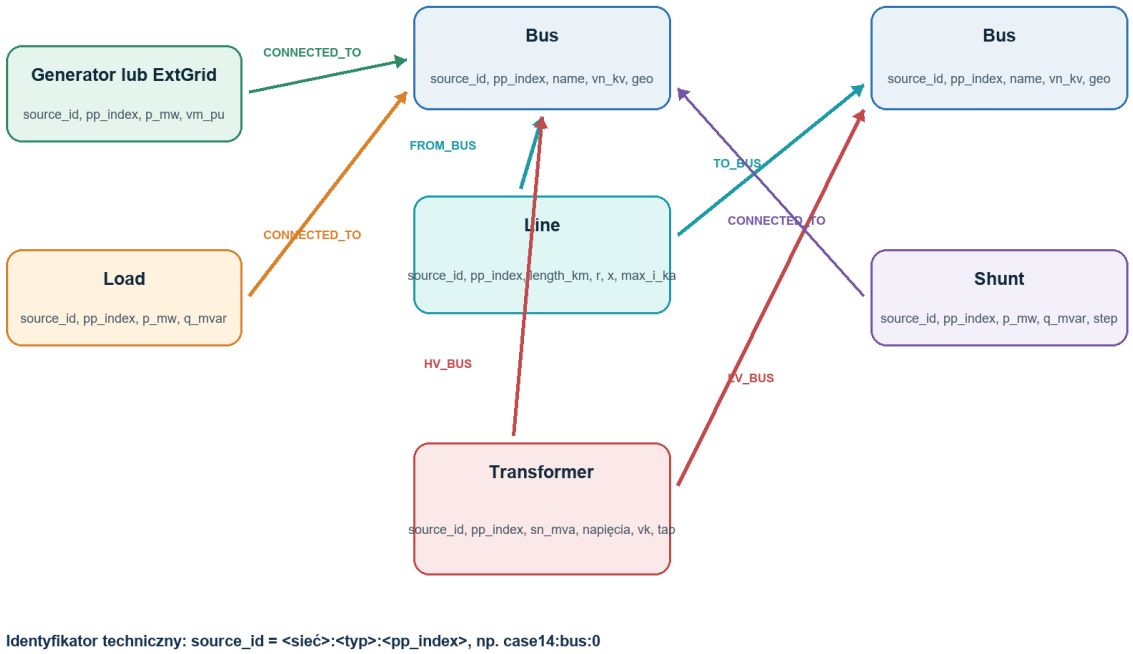
```
{
  "network_id": "example_01",
  "nodes": [
    {"id": "B1", "type": "Bus", "properties": {"vn_kv": 110}},
    {"id": "L1", "type": "Line", "properties": {"length_km": 10}}
  ],
  "relationships": [
    {"type": "FROM_BUS", "from": "L1", "to": "B1"}
  ]
}
```

Schemat odpowiedzi powinien ograniczać wartości pola type do uzgodnionego słownika. Walidacja semantyczna musi dodatkowo sprawdzać istnienie wskazanych obiektów, dopuszczalne kierunki relacji, jednostki oraz wartości wymagane dla poszczególnych typów.

Wstępny model bazy grafowej

Proponowany model traktuje linie i transformatory jako osobne węzły. Dzięki temu ich parametry techniczne pozostają bezpośrednio dostępne, a odwzorowanie jest zbliżone do struktury tabel pandapower. Rozwiązanie upraszcza też ocenę, czy LLM poprawnie rozpoznał konkretny element infrastruktury.

Wstępny model grafu właściwościowego



Rysunek 2 Wstępny schemat węzłów i relacji Neo4j

Identyfikator techniczny może mieć postać source_id = sieć:typ:pp_index, na przykład case14:bus:0. Oryginalny indeks pozostaje dodatkowo dostępny w polu pp_index.

Typy węzłów

Typy węzłów

Etykieta	Znaczenie	Przykładowe właściwości
Bus	Szyna lub węzeł elektryczny	source_id, pp_index, name, vn_kv, type, zone, geo, in_service
Line	Linia elektroenergetyczna	source_id, pp_index, length_km, r_ohm_per_km, x_ohm_per_km, max_i_ka
Transformer	Transformator dwuuzwojeniowy	source_id, pp_index, sn_mva, vn_hv_kv, vn_lv_kv, vk_percent, tap_pos
Generator	Generator sterowany	source_id, pp_index, p_mw, vm_pu, min_q_mvar, max_q_mvar
ExternalGrid	Węzeł bilansujący lub sieć zewnętrzna	source_id, pp_index, vm_pu, va_degree
Load	Odbiór mocy	source_id, pp_index, p_mw, q_mvar, scaling, in_service
Shunt	Element poprzeczny	source_id, pp_index, p_mw, q_mvar, step

Typy relacji

Relacja	Źródło	Cel	Znaczenie
FROM_BUS	Line	Bus	Początkowa szyna linii
TO_BUS	Line	Bus	Końcowa szyna linii
HV_BUS	Transformer	Bus	Strona wysokiego napięcia
LV_BUS	Transformer	Bus	Strona niskiego napięcia
CONNECTED_TO	Generator, Load, ExternalGrid, Shunt	Bus	Przyłączenie urządzenia do szyny

Zakres właściwości jest wstępny. Ostateczną listę należy zatwierdzić po wydrukowaniu pełnych nazw kolumn w używanej wersji pandapower oraz po ustaleniu, które parametry będą oceniane w eksperymentach.

Ograniczenia i zapis w Neo4j

W pierwszej wersji wystarczą ograniczenia unikalności dla source_id w każdej etykiecie. Właściwości wymagane i typy danych można sprawdzać przede wszystkim w warstwie Pydantic, co pozwoli zachować zgodność także w Neo4j Community Edition.

```
CREATE CONSTRAINT bus_source_id IF NOT EXISTS
FOR (n:Bus) REQUIRE n.source_id IS UNIQUE;

CREATE CONSTRAINT line_source_id IF NOT EXISTS
FOR (n:Line) REQUIRE n.source_id IS UNIQUE;

CREATE CONSTRAINT transformer_source_id IF NOT EXISTS
FOR (n:Transformer) REQUIRE n.source_id IS UNIQUE;
```

Import powinien używać MERGE dla elementów identyfikowanych przez source_id oraz transakcji obejmującej spójny fragment sieci. Dane referencyjne i wyniki LLM warto rozdzielić polem graph_variant albo osobnymi bazami, aby przypadkowo ich nie połączyć.

Przykładowe wzorce Cypher

```
MERGE (b:Bus {source_id: $source_id})
SET b += $properties;

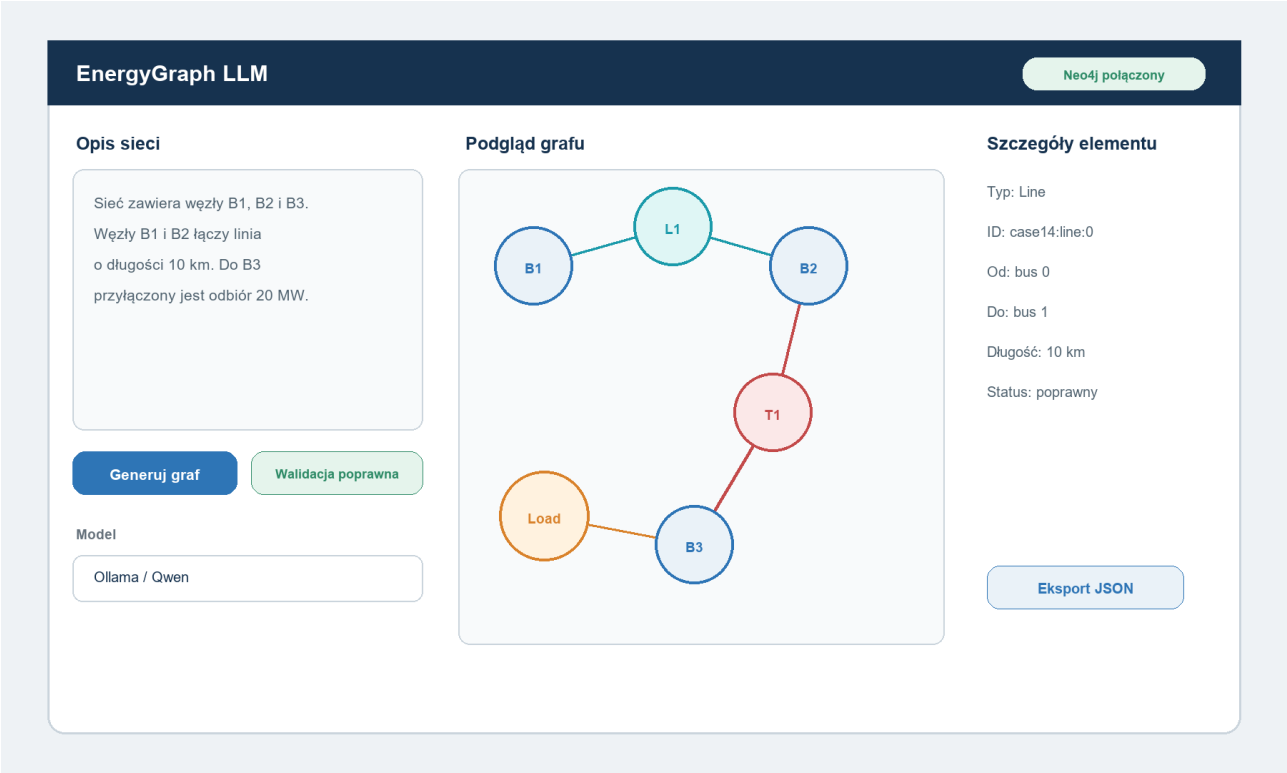
MATCH (l:Line {source_id: $line_id})
MATCH (b:Bus {source_id: $bus_id})
MERGE (l)-[:FROM_BUS]->(b);
```

Otwarte decyzje dotyczące modelu

- Czy zachować osobne etykiety Generator i ExternalGrid, czy wprowadzić wspólną etykietę Source.
- Czy wyniki obliczeń pandapower przechowywać jako właściwości, osobne węzły pomiarowe lub poza grafem pierwszej wersji.
- Czy relacje mają zawierać metadane pochodzenia, pewność ekstrakcji i fragment tekstu źródłowego.
- Czy różne warianty grafu przechowywać w jednej bazie z network_id, czy w osobnych bazach Neo4j.

Potencjalny widok aplikacji

Interfejs pierwszej wersji może składać się z jednego ekranu roboczego. Użytkownik wprowadza opis, wybiera model i uruchamia generowanie. System pokazuje wynik walidacji, graf oraz właściwości wybranego elementu. Dodatkowe funkcje mogą obejmować eksport JSON, porównanie z grafem referencyjnym i listę wykrytych błędów.



Rysunek 3 Robocza makieta głównego widoku aplikacji

Obszar widoku	Zakres funkcji
Panel wejściowy	Opis tekstowy, wybór modelu, parametry promptu i uruchomienie procesu.
Podgląd grafu	Węzły, relacje, filtrowanie typów i zaznaczanie elementów.
Panel szczegółów	Właściwości wybranego węzła lub relacji oraz status walidacji.
Wynik procesu	Lista ostrzeżeń, błędów, czas odpowiedzi i możliwość eksportu.

Plan ewaluacji

Ewaluacja powinna oddzielać poprawność struktury od poprawności właściwości. Sam fakt utworzenia prawidłowego JSON nie oznacza, że graf wiernie odtwarza opis. Dla każdego testu należy zachować tekst wejściowy, oczekiwany graf, odpowiedź modelu, wynik walidacji i listę różnic.

Poziom	Metryka	Interpretacja
Węzły	precision, recall, F1	Czy model wykrywa właściwe elementy i nie dodaje nieistniejących.
Relacje	precision, recall, F1	Czy połączenia, kierunki i typy relacji są poprawne.
Właściwości	exact match lub błąd liczbowy	Czy wartości i jednostki odpowiadają opisowi.
Walidacja	odsetek poprawnych odpowiedzi	Jak często wynik przechodzi schemat bez naprawy.
Stabilność	zgodność między powtórzeniami	Jak bardzo wynik zależy od losowości modelu.
Efektywność	czas, tokeny, koszt	Jaki jest koszt uzyskania pojedynczego grafu.

Proponowane warianty eksperymentów

- Opis prosty, w którym każde zdanie przedstawia jeden obiekt albo relację.
- Opis z parametrami technicznymi, jednostkami i kilkoma obiektami w jednym zdaniu.
- Opis niepełny lub niejednoznaczny, wymagający zgłoszenia braków zamiast zgadywania.
- Porównanie zero shot, few shot oraz structured outputs.
- Porównanie co najmniej jednego modelu lokalnego i jednego modelu przez API.
- Powtórzenie testów dla kilku ustawień temperatury i losowych uruchomień.

Kategorie błędów

Kod	Kategoria	Przykład
E1	Pominięcie obiektu	Brak transformatora opisanego w tekście
E2	Obiekt nadmiarowy	Generator niewystępujący w opisie
E3	Błędny typ	Linia rozpoznana jako transformator
E4	Błędna relacja	Nieprawidłowa szyna początkowa
E5	Błędna właściwość	Niepoprawna długość lub moc
E6	Błąd jednostki	MW zinterpretowane jako kW
E7	Błąd formatu	Odpowiedź niezgodna z JSON Schema

Plan dalszych prac

Etap	Zakres	Rezultat
1 Analiza danych	Pełny wydruk kolumn case14, wybór pól i przypięcie wersji pandapower	Specyfikacja mapowania
2 Model Neo4j	Etykiety, relacje, source_id, constraints i reguły walidacji	Zatwierdzony schemat
3 Graf referencyjny	Konwerter DataFrame do Neo4j oraz testy integralności	Referencyjna sieć case14
4 Integracja LLM	Adapter modeli, prompt, JSON Schema i modele Pydantic	Pierwsza ekstrakcja z tekstu
5 Aplikacja	API, ekran roboczy, wizualizacja i eksport	Prototyp demonstracyjny
6 Ewaluacja	Zbiór opisów, porównanie modeli i analiza błędów	Wyniki do rozdziału badawczego
7 Rozszerzenie	Dodatkowe sieci, dane autentyczne lub fine tuning	Opcjonalny zakres końcowy

Najbliższe zadania

- Zapisać pełne listy kolumn dla tabel case14 i określić minimalne właściwości każdego typu.
- Zatwierdzić reprezentowanie linii i transformatorów jako osobnych węzłów.
- Przygotować klasy Pydantic wspólne dla importu referencyjnego i wyniku LLM.
- Uruchomić Neo4j lokalnie i utworzyć constraints.
- Zaimplementować import szyn, a następnie kolejnych elementów wraz z relacjami.
- Przygotować pierwszy opis tekstowy odtwarzający wybrany fragment case14.

Pytania do promotora

Spotkanie powinno doprowadzić przede wszystkim do zatwierdzenia zakresu pierwszej wersji. Poniższe pytania porządkują decyzje, które wpływają na implementację i późniejszą ewaluację.

1. Czy przedstawiony zakres prototypu jest wystarczający dla pracy inżynierskiej?

2. Czy linie i transformatory powinny pozostać osobnymi węzłami grafu?

3. Które parametry techniczne muszą znaleźć się w minimalnym schemacie pierwszej wersji?

4. Czy case14 wystarczy do demonstracji, jeśli ewaluacja obejmie dodatkowe sieci testowe?

5. Czy prototyp ma wykonywać obliczenia rozpływu mocy, czy skupić się na ekstrakcji, grafie i analizie topologii?

6. Ile modeli oraz strategii promptowania należy porównać, aby część eksperymentalna była wystarczająca?

7. Czy wymagane są autentyczne dane infrastrukturalne, czy wystarczą sieci testowe i opisy generowane na ich podstawie?

Proponowany wynik spotkania

- Zatwierdzony zakres prototypu i rola sieci case14.
- Zatwierdzony wstępny schemat Neo4j oraz lista elementów do pierwszego importu.
- Ustalona liczba modeli, wariantów promptowania i sieci testowych w ewaluacji.
- Decyzja, czy obliczenia elektroenergetyczne należą do zakresu implementacji.

Podsumowanie

Koncepcja systemu jest już określona na poziomie przepływu danych i podziału odpowiedzialności. Najważniejszym elementem pierwszej implementacji będzie wspólny model domenowy używany zarówno przez konwerter pandapower, jak i ścieżkę LLM. Pozwoli on porównywać wyniki bez dodatkowych transformacji oraz precyzyjnie klasyfikować błędy.

Rozpoczęcie od case14 ogranicza złożoność i zapewnia znany punkt odniesienia. Po zbudowaniu grafu referencyjnego kolejne prace mogą koncentrować się na jakości ekstrakcji, niezawodności walidacji i porównaniu modeli. Dodatkowe sieci i dane rzeczywiste powinny zostać włączone dopiero po ustabilizowaniu schematu oraz procesu ewaluacji.

Źródła

1. Dokumentacja pandapower, Power System Test Cases, case14, https://pandapower.readthedocs.io/en/v3.5.6/networks/power_system_test_cases.html
2. Repozytorium pandapower, https://github.com/e2nIEE/pandapower
3. Neo4j, Graph database concepts, https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/
4. Neo4j Cypher Manual, Create constraints, https://neo4j.com/docs/cypher-manual/current/schema/constraints/create-constraints/
5. Materiały robocze i wyniki analizy pandapower case14 przekazane w ramach przygotowania pracy.